



Functional Programming

Looping Without Loops

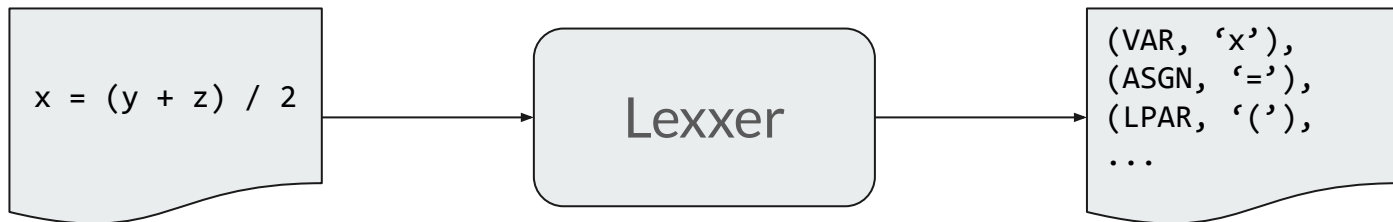
Markos Vassilis, Mediterranean College, 2024 - 2025, Spring

Week 22

Scanning

Scanning / Lexxing

- You are given a Python script, say `foo.py`, which you are asked to execute.
- What is the first thing you should do?
 - Read it!
- The process of reading, i.e., making sense of the actual contents of the source code is often called scanning or lexing (< lex < λέξη).
- The output of a scanner / lexer is, typically, a list of tokens, so you can also silently read “scanning” as “tokenisation”.



Parsing



- Assume that you have now lexed your source code, which means that you have a list of tokens that might make more sense to you. What's next?
 - You have to actually make sense of those, by capturing the subtle structure of the source code.
 - This process is called **Parsing**.
- Put more formally, parsing is the process of creating a data structure (typically an **Abstract Syntax Tree**, AST) that captures the syntactic structure of the underlying code.
- Often, ASTs are called simply Syntax Trees or, even, Trees.

Parsing Example

- Consider this piece of code:

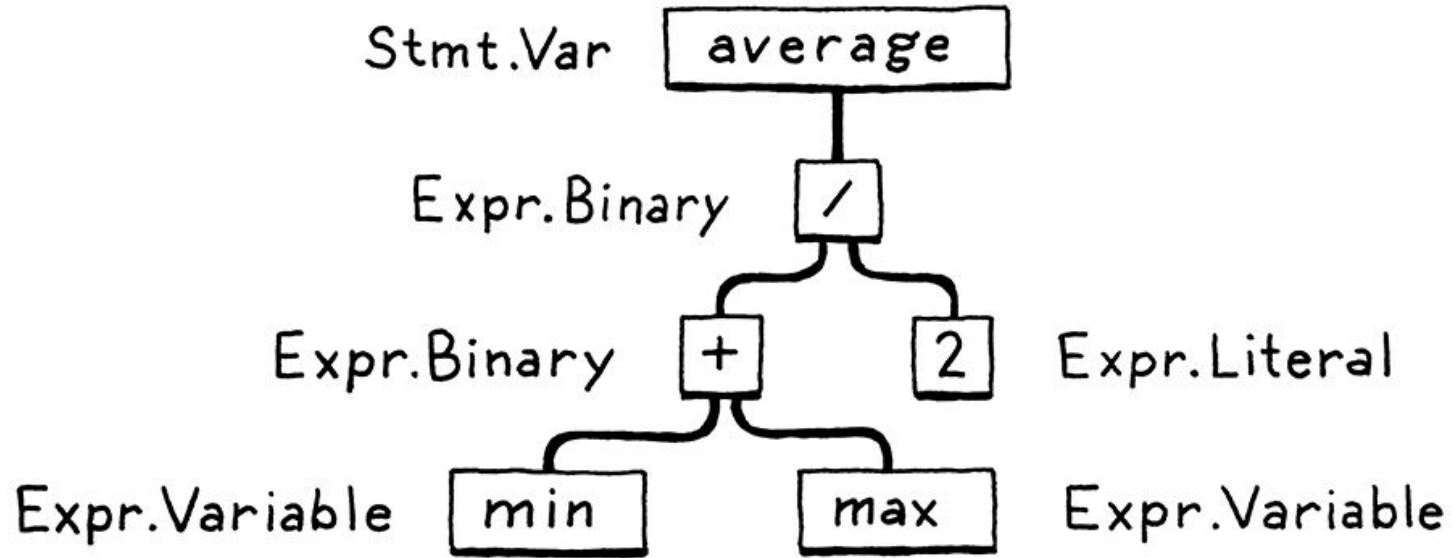
```
var average = (min + max) / 2 ;
```

- How would you expect the lexxed version of this to look like?
- Most probably, something like that:

```
var average = ( min + max ) / 2 ;
```

- What about the corresponding AST?

Parsing Example



Regular Expressions



- What is a regular expression?
 - Roughly speaking, a simple enough expression that can be understood by a silly machine that can only parse one character at a time and make a decision based on the value of that character.
 - Such machines are called DFA (Deterministic Finite Automata).
 - **Not all expressions are regular.**
 - Even more roughly speaking, regular expressions correspond to things that can be expressed and evaluated using finite memory.
 - For instance, the expressions of the form “n 0s followed by n 1s” are not regular.

Regular Expressions + Python



- RegEx are a powerful pattern matching tool in some cases.
- For instance, we can match all sequences of the form “two lowercase letters followed by a digit” just by writing:

`"[a-z][a-z]\d"gm`

- However, with RegEx we cannot perform all scanning / parsing tasks we need to.
- So, feel free to use them but do not expect to solve everything.
- Further reading:
 - <https://docs.python.org/3/howto/regex.html>
 - <https://realpython.com/regex-python/>
 - <https://regex101.com/>

Useful Tools: Buffering



- Assume that you are scanning a ppython file character by character.
- Imagine, also, that you want to scan for the **def** ppython keyword:
 - How can you spot that, since it comprises of three characters?
- Evidently, you need some sort of short-term-memory.
- Such things are, typically, called buffers, and can be as simple as an array or a linked list, a cyclic array implementing a queue, or whatever works best for you.
- Keep that in mind for today's lab...

Useful Tools: Peeking



- Suppose you need to parse the keyword **def** again.
- So, you have a buffer that looks like:

['d', 'e', 'f']

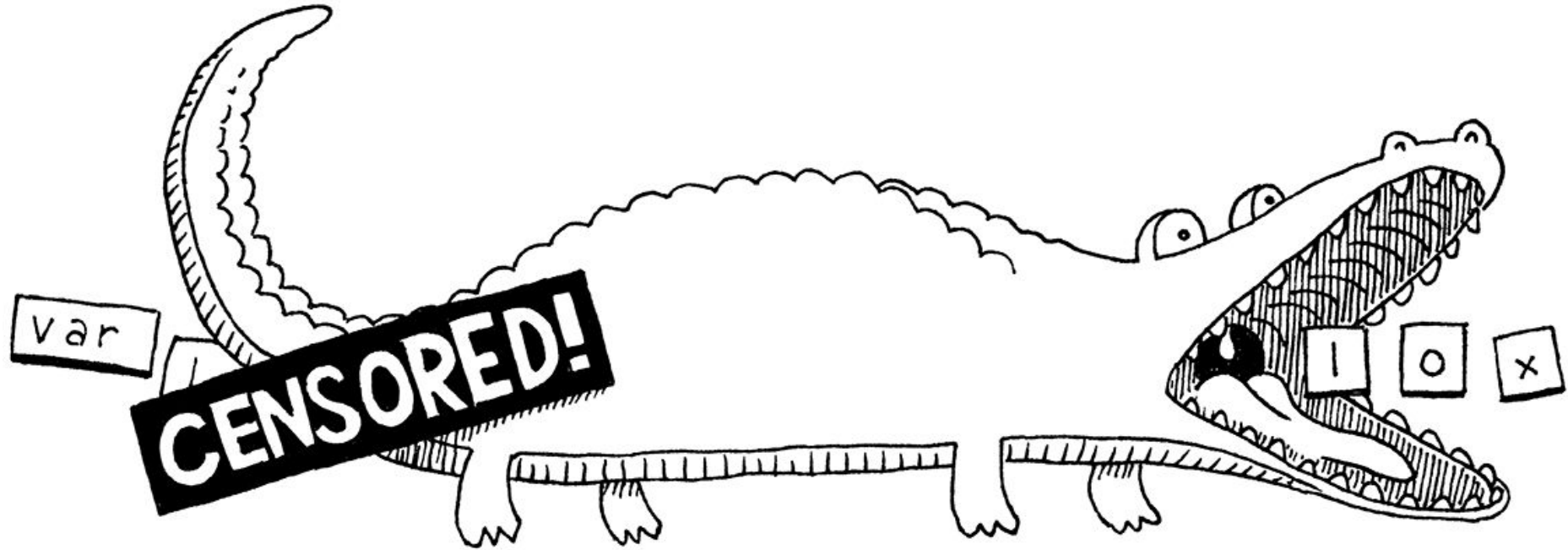
- Are you done?
- What about, e.g., deficit? defence? definite? defghij?
- There are many identifiers that may (or may not) match a certain pattern.
- Thus, aside a **lookbehind** mechanism such as a buffer, we also need a **lookahead** one.
- We often refer to this process as **peeking**.

Maximal Munches



- In the previous case, we have to make sure what we are reading is just **def** and not any longer identifier prefixed with *def*.
- This is a common pattern in scanning, where, in any case we have a dilemma (or tri-lemma?) between two literals such that one is a prefix of the other, the longest one wins.
- This is also called the **maximal munch principle**.
- In the same spirit, we interpret `>=` as such and not as two separate operators, `>` and `=`.
- You will find this idea useful when scanning for reserved words of ppython.

Have Fun Scanning!



ASTs

Beyond Tokens...



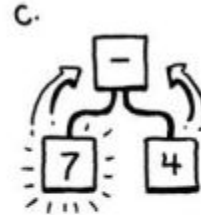
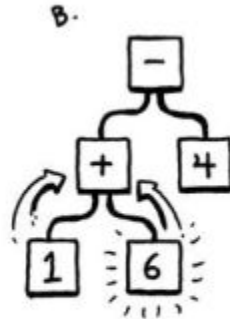
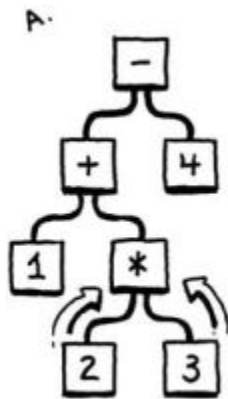
- We have generated a list of tokens from a source file so far.
- However, this is just the first step in the process of meaning extraction from a source file.
- However, a tokens list does not unveil any underlying structure.
 - For instance, you cannot spot syntax errors just by inspecting such a list.
 - Moreover, you cannot make sense of what the code does just by inspection.
- So, we have to find a way to uncover the hidden structure of our code files.
- One way to do so is to construct an **Abstract Syntax Tree (AST)**.

ASTs



- Consider this math expression:
 - $1 + 2 * 3 - 4$
- How would you evaluate it?
 - Start by multiplication, so $2 * 3 == 6$
 - Then add $1 + 6 == 7$
 - Finally, $7 - 4 == 3$.
- Silently, while making the above computations, we are also unwrapping the tree-structure implied by the provided expression.
 - *See next slide for a nice tree-based computation of the above.*

$$1 + 2 * 3 - 4$$



Making Sense Of Stuff



- of sense you how sentence make this do?
 - ???
 - What does the above mean?
 - Well, nothing, most probably.
- How do you make sense of this sentence?
 - You use the English language's grammar to understand how the words used relate with each other in terms of meaning.
- But, what are grammars?
 - Sets of rules that tell us how the building blocks of a language can relate with each other to form meaningful sentences.

Context-Free Grammars



- There are various categories of grammars, depending on how their constructs relate with one another.
- A **CFG** is a **formal grammar** used to describe the syntax of a language.
 - Think of it as a set of rules that define how sentences or strings can be formed in that language.
 - It's called "context-free" because the application of a rule doesn't depend on the surrounding context.
- You can roughly think of CFGs as a simplified version of natural languages grammars.

CFG Main Components



- **Terminals:**
 - These are the basic symbols or "letters" of the language.
 - They are the symbols that actually appear in the strings generated by the grammar.
 - Think of them as the words or punctuation marks in a sentence.
 - Terminals are usually represented by lowercase letters, digits, or symbols like +, -, *, /, etc.
- **Start Symbol:**
 - This is a special non-terminal that represents the top-level structure of the language.
 - It's the non-terminal from which all derivations begin.
 - It's usually designated as the first non-terminal defined in the grammar.

CFG Main Components



- **Non-terminals:**
 - These are symbols that represent grammatical categories or syntactic variables.
 - They are placeholders for more complex structures.
 - They don't appear in the final strings generated by the grammar, but they are used in the rules to define the structure of the language.
 - Non-terminals are usually represented by uppercase letters or descriptive names enclosed in angle brackets (e.g., <Sentence>, <NounPhrase>, <Verb>).

CFG Main Components



- **Production Rules** (or simply "rules"):
 - These are the heart of the grammar.
 - They **specify how non-terminals can be replaced** by other non-terminals, terminals, or a combination of both.
 - A rule has a left-hand side (LHS) consisting of a single non-terminal and a right-hand side (RHS) consisting of a sequence of terminals and/or non-terminals.
 - The LHS is connected to the RHS by an arrow, often represented as $\rightarrow$ or $::=$. For example, the rule $\langle \text{Sentence} \rangle \rightarrow \langle \text{NounPhrase} \rangle \langle \text{VerbPhrase} \rangle$ means that a sentence can be formed by combining a noun phrase and a verb phrase.

A Simple Example (English)



```
<Sentence> -> <NounPhrase> <VerbPhrase>
<NounPhrase> -> <Article> <Noun>
<NounPhrase> -> <Noun>
<VerbPhrase> -> <Verb> <NounPhrase>
<VerbPhrase> -> <Verb>
<Article> -> "the"
<Noun> -> "cat"
<Noun> -> "dog"
<Verb> -> "chases"
<Verb> -> "sleeps"
```

A Simple Example (English)



```
<Sentence> -> <NounPhrase> <VerbPhrase>
<NounPhrase> -> <Article> <Noun>
<NounPhrase> -> <Noun>
<VerbPhrase> -> <Verb> <NounPhrase>
<VerbPhrase> -> <Verb>
<Article> -> "the"
<Noun> -> "cat"
<Noun> -> "dog"
<Verb> -> "chases"
<Verb> -> "sleeps"
```

- **Terminals?**
 - "the", "cat", "dog", "chases", "sleeps"
- **Non-terminals?**
 - <Sentence>, <NounPhrase>, <VerbPhrase>, <Article>, <Noun>, <Verb>
- **Start-symbol?**
 - <Sentence>

“the cat chases the dog”



```
<Sentence> -> <NounPhrase>  
<VerbPhrase>  
<NounPhrase> -> <Article> <Noun>  
<NounPhrase> -> <Noun>  
<VerbPhrase> -> <Verb> <NounPhrase>  
<VerbPhrase> -> <Verb>  
<Article> -> "the"  
<Noun> -> "cat"  
<Noun> -> "dog"  
<Verb> -> "chases"  
<Verb> -> "sleeps"
```

1. <Sentence>
2. <NounPhrase> <VerbPhrase> (using <Sentence> ->
<NounPhrase> <VerbPhrase>)
3. <Article> <Noun> <VerbPhrase> (using <NounPhrase> ->
<Article> <Noun>)
4. "the" <Noun> <VerbPhrase> (using <Article> -> "the")
5. "the" "cat" <VerbPhrase> (using <Noun> -> "cat")
6. "the" "cat" <Verb> <NounPhrase> (using <VerbPhrase> ->
<Verb> <NounPhrase>)
7. "the" "cat" "chases" <NounPhrase> (using <Verb> -> "chases")
8. "the" "cat" "chases" <Article> <Noun> (using <NounPhrase> ->
<Article> <Noun>)
9. "the" "cat" "chases" "the" <Noun> (using <Article> -> "the")
10. "the" "cat" "chases" "the" "dog" (using <Noun> -> "dog")

Dataclasses

Making A Point?



How would you represent a 2D point in Python?

```
class Point:
    def __init__(self, x: int, y: int):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Point(x={self.x}, y={self.y})"

    def __eq__(self, other):
        if not isinstance(other, Point):
            return NotImplemented
        return self.x == other.x and self.y == other.y
```

Too Much For Too Little...



- We have written so many lines of code just to define a simple 2D point.
- But, there is a better way in Python...
- Dataclasses!

```
from dataclasses import dataclass
```

```
@dataclass  
class Point:  
    x: int  
    y: int
```

- Where are the rest declarations we made?

Haskell Has ADTs; Python Has Dataclasses



Dataclasses, introduced in Python 3.7, are classes decorated with `@dataclass` from the `dataclasses` module. This decorator automatically generates several "boilerplate" methods that are commonly needed in classes that represent data. These methods include:

- `__init__`: Initializes the object with the specified attributes.
- `__repr__`: Provides a readable string representation of the object.
- `__eq__`: Enables comparison for equality between instances.
- `__ne__`: Enables comparison for inequality between instances.
- `__lt__`, `__le__`, `__gt__`, `__ge__`: (Optional, generated if `order=True` is specified) Enable comparison operators.
- `__hash__`: (Generated if the class is immutable or if `unsafe_hash=True` is specified) Allows instances to be used in sets and as dictionary keys.

Key Features



- **Decorator:** You use the `@dataclass` decorator directly above your class definition.
- **Type Hints:** You define the attributes of your dataclass with type hints. While not strictly enforced at runtime, these type hints are crucial for the dataclass to generate the correct `__init__` method and for static analysis tools.
- **Generated Methods:** As mentioned earlier, the decorator automatically generates several useful methods.

Key Features



- **Default Values:** You can provide default values for attributes:

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Person:
```

```
    name: str
```

```
    age: int = 0
```

```
    city: str = "Unknown"
```

```
person1 = Person("Alice", 30)
```

```
person2 = Person("Bob")
```

```
print(person1)    # Output: Person(name='Alice', age=30, city='Unknown')
```

```
print(person2)    # Output: Person(name='Bob', age=0, city='Unknown')
```

Key Features



- **Mutability:** By default, dataclass instances are mutable (their attributes can be changed after creation). You can make them immutable by setting `frozen=True` in the `@dataclass` decorator:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class ImmutablePoint:
    x: int
    y: int

point = ImmutablePoint(1, 2)
# point.x = 5  # This will raise an AttributeError
```

Key Features



- **Ordering:** If you need comparison operators (<, <=, >, >=) to be generated, you can set `order=True` in the decorator. This will generate `__lt__`, `__le__`, `__gt__`, and `__ge__` methods that compare instances lexicographically based on their attributes.

```
from dataclasses import dataclass

@dataclass(order=True)
class DataItem:
    priority: int
    value: str

item1 = DataItem(2, "B")
item2 = DataItem(1, "A")
print(item1 > item2)  # Output: True (because 2 > 1)
```

Key Features



- **__post_init__**: You can define a special method called **__post_init__** that will be executed after the automatically generated **__init__** method.

```
from dataclasses import dataclass

@dataclass
class Rectangle:
    width: float
    height: float

    def __post_init__(self):
        if self.width <= 0 or self.height <= 0:
            raise ValueError("Width and height must be positive")
        self.area = self.width * self.height

rect = Rectangle(5, 3)
print(rect.area)  # Output: 15.0
# rect_invalid = Rectangle(-1, 2)  # This will raise a ValueError
```

Key Features



- **Inheritance:** Dataclasses can inherit from other dataclasses or regular classes. The attributes of the parent class will be included in the child dataclass.

```
from dataclasses import dataclass
```

```
@dataclass
class Shape:
    color: str
```

```
@dataclass
class Circle(Shape):
    radius: float
```

```
circle = Circle("red", 5.0)
print(circle)  # Output: Circle(color='red', radius=5.0)
```

Key Features



- **Field Customization:** You can customize individual fields using the `field()` function from the `dataclasses` module. This allows you to control aspects like whether a field is included in the `__repr__`, `__eq__`, or `__init__` methods, or to specify a default factory function.

```
from dataclasses import dataclass, field
from typing import List

def create_id():
    import uuid
    return str(uuid.uuid4())

@dataclass
class Task:
    description: str
    priority: int = 1
    completed: bool = False
    id: str = field(default_factory=create_id, init=False, repr=False,
compare=False)
    tags: List[str] = field(default_factory=list)

task1 = Task("Buy groceries")
task2 = Task("Write report", priority=2, tags=["urgent"])
print(task1)  # Output: Task(description='Buy groceries', priority=1,
completed=False)
print(task2)  # Output: Task(description='Write report', priority=2,
completed=False, tags=['urgent'])
print(task1.id)  # Output: A unique UUID
```

Okay, But Why Not Classes?



Use Dataclasses:

- When you need to create classes primarily for holding data.
- When you want to avoid writing repetitive `__init__`, `__repr__`, `__eq__`, etc., methods.
- When you need a clear and concise way to define data structures.
- When you want the benefits of type hints for your data classes.

Don't use Dataclasses:

- When your class has significant behavior and logic beyond just storing data. In such cases, a regular class might be more appropriate.
- When you need very fine-grained control over the generated methods. While `field()` provides some customization, for complex scenarios, manual implementation might be necessary.

Fun Time!

Do You Read Books?



In case you have not fully implemented last week's lab, you can find the instructions here:

`./labs/fp_css_parser_lab_01.pdf`

As this is relevant to your assignment, take care to work on all tasks, especially the ones related to the actual Goodreads dataset.

Any Questions



Don't forget to fill in the questionnaire shown right!



<https://forms.gle/YU1mjxjBjBf8hyvH7>